

BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER GRAPHICS AND MULTIMEDIA
ÚSTAV POČÍTAČOVÉ GRAFIKY A MULTIMÉDIÍ

ADVANCED INJURY SIMULATION OF GAME CHARACTERS

POKROČILÁ SIMULACE ZRANĚNÍ HERNÍCH POSTAV

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **ADAM HELEŠIC**
AUTOR PRÁCE

SUPERVISOR **Ing. TOMÁŠ CHLUBNA, Ph.D.**
VEDOUCÍ PRÁCE

BRNO 2026

BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

Bachelor's Thesis Assignment

172352

Institut:	Department of Computer Graphics and Multimedia (DCGM)
Student:	**Helešic Adam**
Programme:	Information Technology
Title:	**Advanced Injury Simulation of Game Characters**
Category:	Computer Graphics
Academic year:	2025/26

Assignment:

1. Familiarize yourself with game development in the Godot engine.
2. Study the issue of game character animation and damage during combat.
3. Research existing and possible solutions to the given problem.
4. Design a user interface and the final functionality of an application that demonstrates advanced character reactions to injuries.
5. Implement the application and demonstrate its use with different types of data.
6. Perform user or performance testing of the results.
7. Document and publish the final application.
8. Create a video representing the results of your work.

Literature:

- Adams, Ernest, and Joris Dormans. Game mechanics: advanced game design. New Riders, 2012. ISBN 0321820274, 9780321820273.
- Geijtenbeek, T., Vasilescu, D., Egges, A. Injury Assessment for Physics-Based Characters. In: Allbeck, J.M., Faloutsos, P. (eds) Motion in Games. MIG 2011. Lecture Notes in Computer Science, vol 7060. Springer, Berlin, Heidelberg. 2011. DOI: 10.1007/978-3-642-25090-3_7.
- Francis Laclé, Nicolas Pronost. Real-Time Musculoskeletal Model for Injury Simulation on 3D Human Characters. Motion In Games Conference (MIG), Nov 2012, Rennes, France.
- Aristidou, A., Lasenby, J., Chrysanthou, Y. and Shamir, A. Inverse Kinematics Techniques in Computer Graphics: A Survey. Computer Graphics Forum, 37: 35-58. 2018. DOI: 10.1111/cgf.13310.

Requirements for the semestral defence:
Items 1-4, experiments leading to item 5.

Detailed formal requirements can be found at https://www.fit.vut.cz/study/theses/

Supervisor:	**Chlubna Tomáš, Ing., Ph.D.**
Head of Department:	Černocký Jan, prof. Dr. Ing.
Beginning of work:	1.11.2025
Submission deadline:	13.5.2026
Approval date:	7.11.2025

Abstract
todo

Abstrakt
Do tohoto odstavce bude zapsán výtah (abstrakt) práce v českém (slovenském) jazyce.

Keywords
Godot, Godot Engine, GDScript, injury system, behavior tree, enemy AI, inverse kinematics, mesh deformation, game design, game development

Klíčová slova
Godot, Godot Engine, GDScript, systém zranění, behaviorální strom, AI nepřátel, inverzní kinematika, deformace meshe, herní design, herní vývoj

Reference
HELEŠIC, Adam. *Advanced Injury Simulation of Game Characters.* Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Tomáš Chlubna, Ph.D.

Advanced Injury Simulation of Game Characters

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Ing. Tomáš Chlubna, Ph.D. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used Cursor to correct spelling and other language mistakes and when working on the software.

.....

Adam Helešic

May 6, 2026

Contents

1 Introduction **2**

2 Game Character Injury Systems and Animation **3**
2.1 Game Design . . . 3
2.2 Case Studies . . . 4
2.3 Locomotion . . . 6
2.4 Enemy AI . . . 9
2.5 Godot . . . 12

3 Injury System Design **14**
3.1 The Modularity of Godot . . . 14
3.2 Body Part Injury System . . . 14
3.3 Enemy AI Based on Body Parts . . . 15
3.4 Knockback . . . 16
3.5 Mesh Deformation . . . 16
3.6 Enemy Design . . . 16
3.7 Player Controller . . . 17

4 Implementation and Demonstration **18**
4.1 Modeling . . . 18
4.2 Behavior Tree . . . 21
4.3 Deformation Shader . . . 26
4.4 Body Part Nodes . . . 27
4.5 Knockback `SkeletonModifier3D` . . . 29
4.6 Player Controller . . . 29
4.7 The Manticore Enemy . . . 30
4.8 Demonstration in Various Test Cases/Situations . . . 33

5 Performance Testing **34**

6 Documentation, Publication, Conclusion **35**

Bibliography **36**

Chapter 1

Introduction

The goal of this thesis is the design and implementation of a dynamic injury system of game characters in the Godot Engine. The term *character* here describes game characters in general, and is not intended to be limited to anthropomorphs. This demo is not player-centric, but rather aims to implement a standalone injury system and its effects on the creature's behavior and animation.

Chapter 2 explores existing injury systems from a design perspective and explains the basics of the mechanisms that are used in the implementation of the final product, which include inverse kinematics (for character animation) and AI patterns (specifically the finite state machine and the behavior tree).

Chapter 3 details the design of the game demo. It explains the design of the enemy model, the subsystems necessary for implementing a cohesive enemy with a specialized injury system.

Chapter 2

Game Character Injury Systems and Animation

Combat and injury have been a part of human history even since prehistoric times. Physical conflict accompanies many stories, large-scale historical events, and even modern times. It is then no wonder that games have used this theme in a myriad of ways. From mythical, heroic combat with swords and magic to competitive shooters, countless games had to contend with designing and implementing combat and injury systems.

Each game has its budget and own set of priorities based on their design. Sometimes, developers may decide to prioritize other parts of the game like visuals, story, and voice acting over the implementation of complex combat systems, opting for greater abstraction. The easiest systems are the ones that model characters as a singular entity with no subparts and a number representing hit points that determines its binary state – alive or dead. However, other designs call for a more complex and varied experience.

This chapter delves into the basics of game design and examines how other games have developed such systems. Next, it looks into the mechanisms constituting an advanced injury system of game characters – systems of simulating harm and injury, locomotion, and enemy artificial intelligence (AI) that reflects the injuries in its decision making.

2.1 Game Design

Game design is the act of deciding what a game should be. Decisions about rules, look and feel, timing, pacing, risk-taking, rewards, punishments, and everything else the player experiences is the responsibility of the game designer [6]. Schell proposes a very player-centric definition – *anything* that the player experiences in the game is part of the game design. It is a fair, but broad explanation. A lower level approach may be more useful in practice when creating a new design.

Adams and Dormans argue that games are built on a set of rules that together define the play experience. There are a few principles to designing rulesets. Firstly, a game should have an extent of unpredictability (but not be too unpredictable nor a predetermined sequence of events). Secondly, the players' skill, strategic decisions, and knowledge of the games should make a difference. The longer the game, the more should skill contribute to victory. Thirdly, complex gameplay facilitated by many interactive parts and interweaving rules adds another layer of unpredictability and creates an opportunity for player skill and game knowledge to matter. The player is incentivized to learn to read and foresee the game

state and understand the strategic complexities to make judgement calls based on these observations. Most games combine these three sources of creating engagement – chance, player choices, and complex rules [1].

The game design in this thesis aims to create a more complex combat experience with less abstractions, where the player can make meaningful choices about where to attack an enemy to cripple its capabilities based on the body part harm. This is the lens through which later sections examine existing implementations in games. While assessing, designing, and implementing these features, the goal is also to develop in-engine tools that would make creating such enemies more streamlined in Godot.

Simulating Harm and Injury

The implementation of injury depends on where the design lands on the spectrum between realism and abstraction for the sake of gameplay. One avenue would be to simulate the game character as a physics-based entity with detailed models of musculature. Injury would be detected precisely where a collision with a weapon happens, and articulated movement that uses the muscles in those parts disabled. However, the goal of this thesis is not to create a perfectly realistic body simulation, but to provide a framework and streamline creating enemies with targetable body parts that serve as prerequisites for enemy abilities, and making it publicly available to Godot users.

2.2 Case Studies

This section examines the systems within games that implement a body part injury system and their approach to the design.

In Horizon Zero Dawn and Horizon Forbidden West, components and armor plates can be removed with high tear damage weapons, which deals little to no damage, but reveals weak points or disables attacks[1].

In Monster Hunter Wilds, removing body parts is a core part of the experience, gathering them as loot or removing the monsters' dangerous abilities. Different weapon types (slashing, blunt, ranged) deal increased damage to specific enemies' body parts. For example, slashing weapons deal extra damage to tails, while blunt weapons can stun, making the enemy roll on the ground for a period of time[2].

Dead Space has many enemies with weak spots, different for each enemy type. Attacking those parts can limit an enemy's attack or movement. Some enemies can have armor that protects their weak points, forcing the player to focus on another weak point or to change their position to access weak points hidden on the enemy's back. Enemies can also have weak points that are dangerous to the player, such as explosive sacks damaging the player when destroyed in close proximity, forcing another angle of strategy to the weak point system. The important aspect of this combat system is in how it enforces different strategies that the player needs to apply based on the enemy type's weak point effects[3].

Fallout offers a slow-motion targeting system that allows players to target specific body parts. Each body part has its own chance of being hit, modified by distance and cover. The

[1] https://www.polygon.com/horizon-zero-dawn-guide-walkthrough/21357205/machines-fight-rockbreaker-behemoth-thunderjaw-stormbird-deathbringer-tallneck-corrupted-cauldron/

[2] https://www.polygon.com/monster-hunter-wilds-guide/528353/monster-tails-fangs-claws-horns-how-to-get-sever-cut-break/

[3] https://www.dualshockers.com/dead-space-remake-necromorph-weakness/



Figure 2.1: First row: Dead Space's necromorph enemies with harmable limbs highlighted red[7]. Second row, left to right: Fallout's V.A.T.S.[8]and Phoenix Point's free aim targeting[9].

number of attacks a player can make is dependent on the player's action points and the weapon type. Body parts have a status that is separate from the target's overall health. A status bar can be worn down until empty, at which point the limb becomes crippled. Crippled limbs have effects like hindered movement, removing a robot's weaponized arm, or shooting an enemy's head (which in most cases kills them outright)[4].

In Xenoblade Chronicles X, the player can target appendages which have their own health pool, separate from the overall one. Destroying appendages disables certain attacks and lowers the enemy's defenses[5].

Phoenix Point's "free aim" combat allows the player to hit specific body parts based on projectile collisions. Each body part has their independent values (like health) and features. Disabling parts has various effects, such as hindering mobility, disabling the enemy's main weapon, or triggering a bleeding effect. Each part also has a different armor value which reduces damage[6].

[4] https://www.videogamer.com/guides/a-beginners-guide-to-combat-in-fallout-4/

[5] https://deltiasgaming.com/xenoblade-chronicles-x-combat-guide/

[6] https://www.rockpapershotgun.com/phoenix-point-free-aim-and-part-damage-guide-how-to-use-the-free-aiming-system-for-more-precise-shots

[7] Image taken from https://vulkk.com/2023/02/03/dead-space-remake-combat-tips-how-to-defeat-each-type-of-necromorph/.

[8] Image taken from https://www.thegamer.com/fallout-3-vats-bethesda-nearly-cut/.

[9] Image taken from https://www.rockpapershotgun.com/phoenix-point-free-aim-and-part-damage-guide-how-to-use-the-free-aiming-system-for-more-precise-shots.

Summary

Body parts most often have a feature bound to them – an ability (like a weapon attack), protection (armor that protects weak points or targetable body parts from being targeted), or mobility. Attacking such body parts hinders those features, e.g., slowing an enemy down or disabling an attack. Additionally, body parts may serve as weak points, granting the player an advantage when being hit in a particular way (like stunning enemies by hitting their heads with a blunt weapon in Monster Hunter Wilds).

There are also variable values that modify the way limbs take damage, such as a hit chance (Fallout) or armor that reduces damage taken (Phoenix Point).

The designs also aim for enemy variance that enforces different strategies to be employed by the player (like in Dead Space).

Game designs should also be mindful of not overwhelming the player. This can be solved by making combat systems more diegetic, meaning embedding mechanics into the "narrative world" (the game world as perceived by the game characters). For example, whichever body part the player shoots, that part takes damage, as opposed to the player choosing from several invasive UI popups.

2.3 Locomotion

Locomotion is the problem of making game characters move. Often, it is not a single system, but an intertwining of AI, gameplay, physics, and animation systems [5, Chapter 25].

This thesis concerns itself mainly with enemy AI and animation. Inverse kinematics is used to make simulated movement look more natural, allowing for end effectors, such as feet, to be placed dynamically based on things like raycast collisions with the terrain mesh (see adjusted foot height in Figure 2.2). Inverse kinematics can also be used for attaching end effectors to a point on the ground, correctly aiming for a target, etc. This can prevent awkward inconsistencies like sliding walk animations or animations when reaching for an object is offset from the actual target due to a static animation.

The injury system has to be dynamic, adapting to whichever injuries are applied to the character's body. Hence procedural animation fits this goal of adaptability well. Rather than traditional animation with preset model movements, procedural animation can account for and adapt to injuries on the run, and make for a more realistic and immersive interaction with the injured character.

Inverse Kinematics

Inverse kinematics is *the rotational and translational motion of points, bodies (objects) and systems of bodies (groups of objects) without consideration of what causes the motion or any reference to mass, force or torque* [2]. In short, it is about moving and rotating a chain of links to reach a specific point.

It had originally been used in robotics for moving arms to targets, but has also expanded into animation in computer graphics. In the latter field, it is very useful for animating *articulated figures* (figures with a specific, solid form like humans, animals, and generally creatures with limbs). These models are often complicated, and require efficient calculation of complex and multi-link models that can have specified constraints (e.g., a maximum angle that a link can turn by). The multi-link models of articulated figures in inverse kinematics



Figure 2.2: Dynamic foot placement using inverse kinematics (The Witcher 3).

are calculated using the positions of the model's joints, known as *end effectors* (often the end joints), their potential constraints, and the target positions to be reached [2].

A *model* is a series of *links* connected by *joints*. Joints are the main component allowing movement – their positions determine the positions and orientations of the links. Links are the "connective tissue" between joints and are defined by their length, shape, volume, and mass. A *posture* is the skeletal configuration of the figure and must most often satisfy constraints to adhere to realistic movement, such as the torque limitations of each joint or the links not intersecting with other models in their path. A skeletal configuration is usually separated into *chains*, standalone parts attached to a static point on the model (e.g., the limbs of animals and anthropomorphs). The chains usually work in a hierarchy, and the movement of one joint affects the movement of the ones before it [2]. An example model can be seen in Figure 2.3. The way the movements of joints affect each other is illustrated in Figure 2.4. The target is reachable in (b) and (c), while (d) showcases the attempt of the chain to reach an unreachable target. The specific way these links bend depends on the concrete inverse kinematics solver implementation.

Usage of Inverse Kinematics

Forward kinematics (traditional animation where known, predefined transformations are applied to the model) can fall short in situations where dynamic adjustment of the target locations may be needed, such as characters standing on slopes or an appendage reaching for a target that can be a variable distance or height away. The bending of the links (such as limbs) is dynamically calculated, making for more natural-looking and adaptive movement, which is a desirable trait for our dynamic body part injury system that needs to account for the injuries made.

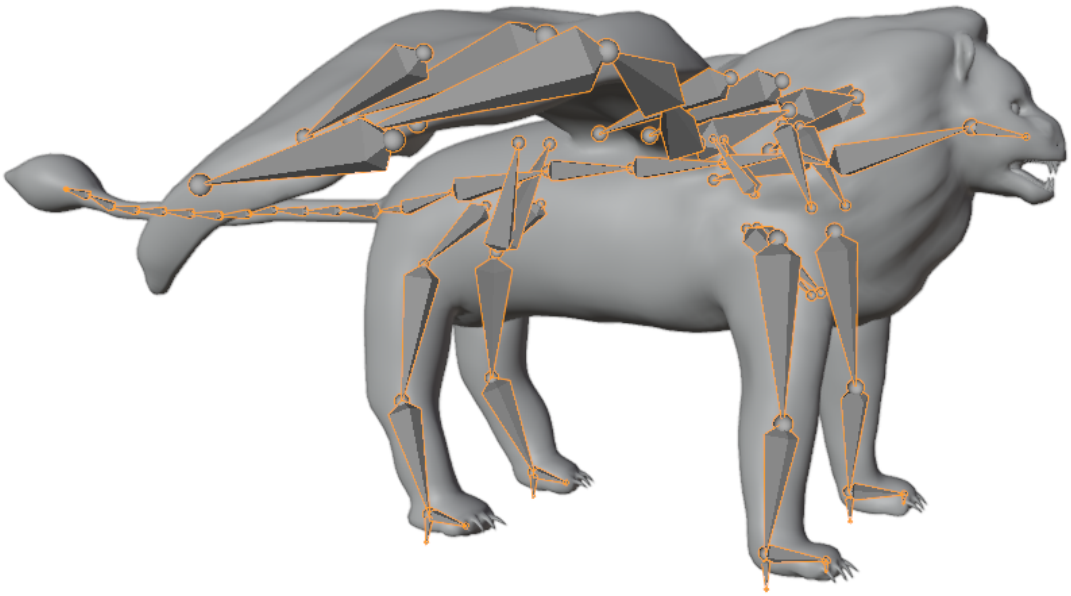


Figure 2.3: An inverse kinematics *model* – a skeleton rig of the game character consisting of *chains* like the limbs, the tail, or the wings.

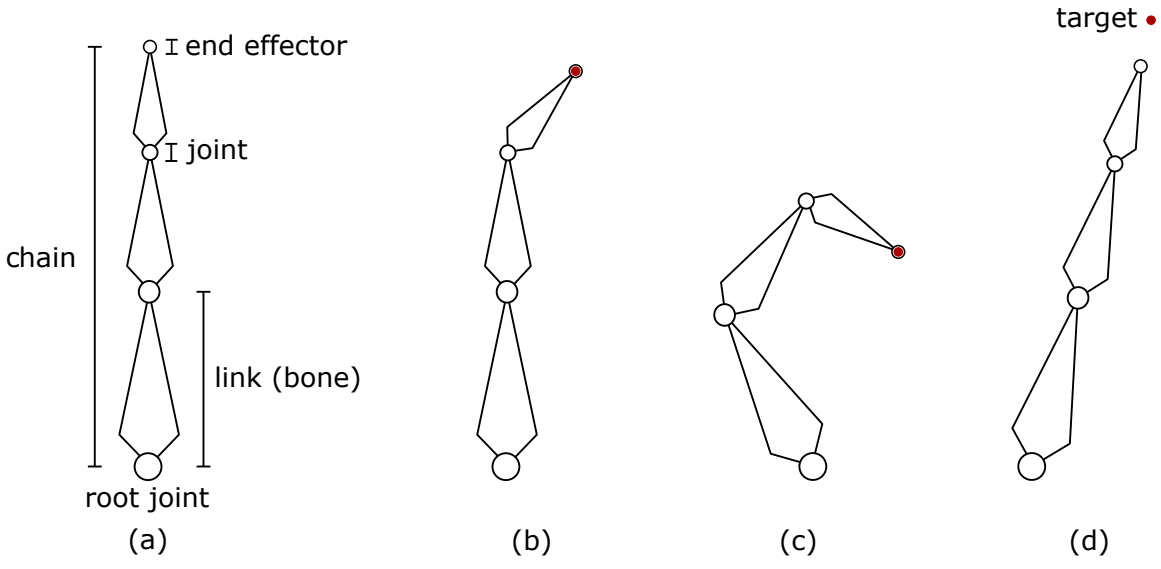


Figure 2.4: An inverse kinematics chain description and examples of the chain bending so that the end effector reaches the red dot target, showing two reachable and one unreachable target point positions.

2.4 Enemy AI

There are many ways to implement decision making in games. This section explains finite state machines, behavior trees, and utility theory.

2.4.1 Finite State Machine

One of the possible implementations of enemy AI is a finite state machine (FSM). FSMs are an architectural structure that can encapsulate enemy behavior into separate states, such as *Patrol* or *Attack*. A central directing *state machine* controls which state is active and allows for the state to be changed. The *state* itself has the encapsulated behavior logic (e.g., a patrolling enemy walking between two points) and the transition logic like methods to be executed when entering or exiting a state [3]. A simple FSM implementation is illustrated in Figure 2.5.

Finite state machines are a quick and easy implementation of enemy AI where the behavior can be separated into states and where there are clear transitions in each state. It can get convoluted if the transition requirements are complex – in the most extreme cases, it would be FSMs where each state can transition to any other state.

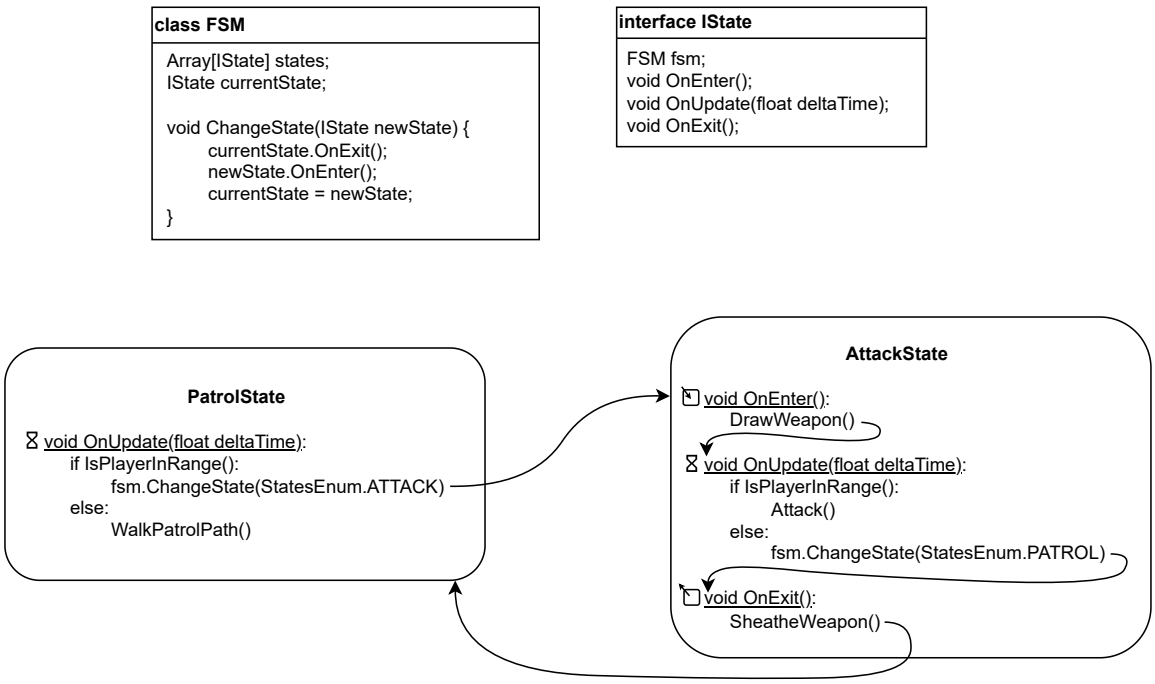


Figure 2.5: An example of a finite state machine implementing a simple enemy AI.

2.4.2 Behavior Tree

Another option for enemy AI implementation is the behavior tree (BT), a way to structure decision making logic into a more streamlined form. BTs follow the traditional node-based architecture of trees, but contain special node types that serve the node traversal and behavior logic. A sample BT and its traversal is illustrated in Figure 2.6.

The node type and implementation descriptions from this section are taken from [5, Chapter 6]. The most commonly used nodes are as follows:

- *Behavior* is an abstract interface that can be activated, run, and deactivated, and passes back a *return status*. The concrete implementations of this interface are *actions* (accessing information and making changes to the world) and *conditions* (a primary way of checking information from the world).
- *Sequences* can be thought of as sequential `AND` gates – each of their children nodes are executed in order one after another, until all have executed successfully or until one of the children fails. Not only is this useful for chaining behavior, but also for setting up conditions for whether to execute this branch at all – if a condition node is added as its first child, it must return `true` for the sequence to continue executing.
- *Selectors* can be thought of as sequential `OR` gates or fallbacks. It executes its children in order until one returns `SUCCESS` or `RUNNING`. If a child fails, the next one is its "fallback" behavior.
- *Parallels* execute all of their child nodes simultaneously. It is important to clearly define these nodes' return states (e.g., returns `SUCCESS` only if all of the child nodes succeed). When exiting, the termination of all child nodes must also be handled cleanly (although in some cases, like animations and sound effects, a delay in the child's termination to after it has finished its action may be desired).

Return statuses are crucial for tree traversal. Most often, behavior nodes return whether they completed their run as expected or not, or that they're still running – `SUCCESS`, `RUNNING`, or `FAILURE`. However, BTs can also leverage the return status for more efficient implementations, such as the `SUSPENDED` status code in event-driven BTs.

This basic BT implementation can then be extended by more specialized nodes.

- *Decorators* (named after the design pattern) allows behaviors to be wrapped in other behaviors, such as repeating a child's behavior n times.
- *Filters* will not execute its children under specific conditions (such as an action cooldown or a target being out of range of an action). Filters can be easily created by attaching a condition to the start of a sequence, as described above. Advanced filters can be created by combining sequence (`AND`), selector (`OR`), and condition nodes.
- *Monitors* serve for continously checking if assumptions for an active behavior are valid, and signal that the whole subtree should be exited when they become invalid. It runs in parallel with other nodes, usually a single sibling that is a behavior subtree that needs some continuous state checking (to ensure it makes sense to keep running it).
- *Active selectors* actively recheck their decisions regularly, allowing them to retry higher priority branches (e.g., interrupting a patrol with a search behavior if the player is spotted).

It is useful to have a `BehaviorTree` object that is built by a `BehaviorTreeBuilder`. This will serve as the central point for storing and updating the tree. The BT has an `update` method which performs the traversal of the tree and is the entry point of the BT. It is called whenever an update is needed, though it need not be every game frame – every other frame or 5 frames per second can suffice (to lighten or spread out the processing load). For a more efficient approach, an event-based node traversal implementation is possible where a central scheduler manages the traversal using the observer pattern.

Actions usually have to interact with other parts of the system. For this purpose, it can fetch data from a blackboard, a shared data storage for the behavior tree. It may also be important to have a shutdown mechanism that handles action exits cleanly [5, Chapter 6].

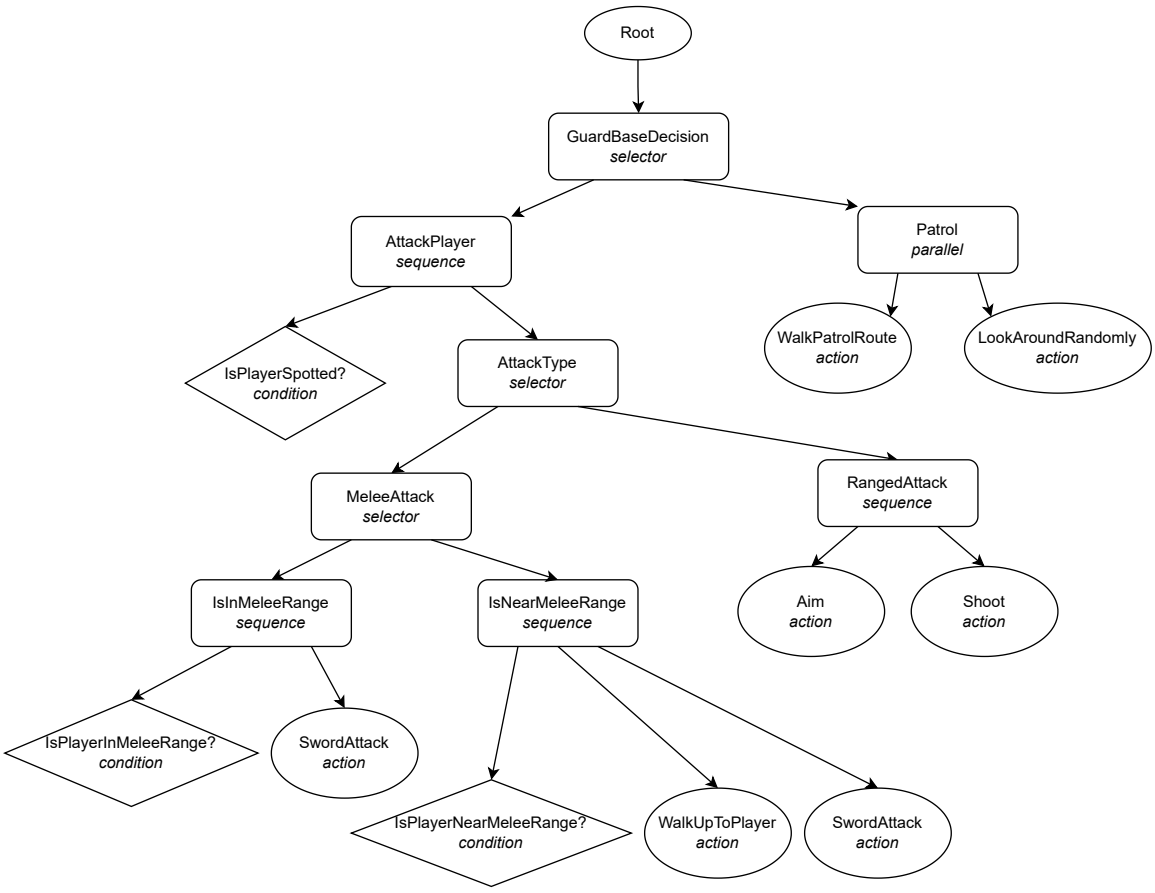


Figure 2.6: An example of an enemy AI behavior tree. A possible traversal in a situation where the player is not seen would go as described: The `GuardBaseDecision` seletor enters `AttackPlayer` (its first child). As a sequence, this node has to run all of its children in order to succeed. The `IsPlayerSpotted?` condition is false, and thus the whole sequence returns the `FAILURE` return state. The `GuardBaseDecision` selector then enters its next child (the fallback), which is a parallel node with two action children. The `WalkPatrolRoute` and the `LookAroundRandomly` actions start simultaneously executing.

2.4.3 Utility

Utility theory in enemy AI design refers to the evaluation of *utility* across multiple choices. Utility is the usefulness of the choice within the given context. The AI calculates a utility score for each action, and the action with the highest score is then chosen. The calculation of utility can be simple if the decision factors are comparable, like only taking into account a price of the same item from two sellers. However, more complex situations require the conversion and revaluation of values from one context to another, unified context. Utility scores should also be consistent and on the same scale, as they are directly compared to each other. Normalized utility scores are a good baseline for consistency [5, Chapter 9].

Utility calculations can also be integrated into behavior tree traversal. The behavior tree implementation can be extended by a *utility selector* node. It first queries its children for a utility score, and then decides which child to enter based on that score. For an AI aimed towards optimal decision making, the child with the highest score would be picked. Alternatively, the choice could be randomized, but still weighted towards the higher scored children [5, Chapter 10].

2.5 Godot

Godot is a game engine capable of creating 2D and 3D games and tools. It exports to desktop, mobile, the web, and even consoles. It comes with an editor with many integrated tools, such as the code editor, an animation editor, a tilemap editor, a shader editor, a debugger, or a profiler. External programs are also supported (e.g., code editing in VSCode with GDScript and C# plugins or direct imports of Blender scenes)[10].

Programs in Godot can be written with the native GDScript (a language akin to Python) or with C#. GDScript is tightly integrated with the editor itself. Certain C# project exports are limited in Godot 4, the current up-to-date version of the engine (Android and iOS are still experimental[11], and web exports are only supported in Godot 3[12]). Other languages (C++, Rust, ...) are supported via GDExtension which allows developers to interact with third-party libraries in the engine at runtime. The engine itself can also be directly edited, as it is free and open source under the MIT license[10].

2.5.1 Key Concepts

Godot's architecture is object-oriented with a flexible scene system and node hierarchy. A game is a tree of nodes grouped into scenes. Games are made up of scenes, which can be as small and simple (a sword, a tree) or as compound (an enemy, an entire level) as necessary, since scenes can be nested inside other scenes. A scene is composed of one or more nodes, which are the smallest building block components arranged into a tree. The engine provides a multitude of node types that handle a lot of the things a game requires. These nodes can be combined or extended, allowing developers to tailor their scenes to their needs[13].

To communicate with each other, nodes use signals, an integrated version of the observer pattern. Instead of hard-wiring nodes to each other, a node can declare and emit signals that other nodes can connect to. For example, buttons emit press signals, areas emit signals when something enters their collision shape, or users can create their own and decide when they are to be emitted[13].

[10] Introduction to Godot, available at: https://docs.godotengine.org/en/stable/getting_started/introduction/introduction_to_godot.html

[11] C#/.NET Scripting, available at: https://docs.godotengine.org/en/stable/tutorials/scripting/c_sharp/index.html

[12] Exporting for the Web, available at: https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_web.html

[13] Key Concepts, available at: https://docs.godotengine.org/en/stable/getting_started/introduction/key_concepts_overview.html

2.5.2 Engine Extensibility

The Godot editor itself runs on the game engine. It can be extended via its native API, using the same language as the game code. It supports hot-reloading, building plugins and extending the editor (using the `@tool` script annotation), as the game code can run directly inside of it[14].

2.5.3 Shading Language

Godot uses a shading language similar to GLSL ES 3.0 for its shaders. Most data types and functions are supported. It also offers in-built variables like `UV`, `COLOR`, `VERTEX`, etc., though what is available depends on the type of shader and the function being used. Uniform variables are parameters that can be passed into the shader and show up in the editor. They can also have hints that tell the editor how they should be displayed and how they can be edited[15].

[14] Godot Design Philosophy, available at: https://docs.godotengine.org/en/stable/getting_started/introduction/godot_design_philosophy.html

[15] Godot's Shading Language, available at: https://docs.godotengine.org/en/stable/tutorials/shaders/shader_reference/shading_language.html

Chapter 3

Injury System Design

The end goal of this thesis is to create a demo of an enemy system that allows players to target different body parts and destroy them, which affects the enemy's behavior. A secondary objective is to build up supporting structures and tools that help modularly construct body-part-based enemies in Godot.

3.1 The Modularity of Godot

Godot's scene and node-based architecture and its tooling API allow for great extensibility. It is possible to create classes by using the `class_name` keyword in a script, which also makes them a custom node type that can be inserted into the scene tree and they will work just like native node types (such as `CharacterBody3D`, `Camera3D`, `CollisionShape3D`, etc.). It is even possible to create tool scripts (by using the `@tool` annotation at the top of a script) which run live in the editor. The scripts' variables can be prepended with `@export` which makes the values visible and editable directly in the engine's editor. Their visibility can even be toggled or the possible enum values supplied based on the values of other variables. Via these engine features, Godot provides a toolbox for developing modular, extensible, and encapsulated systems that can then be woven together into a more complex scene.

3.2 Body Part Injury System

The goal is to develop an injury system with nodes and tooling to facilitate creating enemies with body-part based models and abilities like in the video games in Chapter 2.

Losing a body part needs to have a meaningful effect on the gameplay, as the player should be able to make choices, the results of which impact the state of the game [6]. For example, a player that prefers melee combat might target an enemy's wings to prevent it from flying and getting out of melee range or to first disable its fly-and-dive attack which is hard to avoid. Such a system provides dynamic reactivity to the player's actions and choices.

The game combines the three sources of unpredictability mentioned in Section 2.1 that make for more engaging systems. The element of chance comes from the enemy's AI, which makes randomized decisions in the behavior tree. The player choice lies in which body parts the player attacks so that they can disable the enemy's abilities that require those parts to be triggered. The complexity comes from the way body parts serve as prerequisites for the enemy's abilities, such as two wings being necessary for flight, at least three limbs for

running, or a stinger tail for ranged attacks, and the way they facilitate and limit what actions it has available in its behavior.

A central `Body` node stores the total health, with `BodyPart` child nodes that either have their own health or subtract health from the `Body`.

The game also has visual effects and visual feedback to the player's actions. While it has the practical importance of communicating to the player (like whether an attack has hit the enemy or that the player has taken damage), it also serves to make the core of the system – the dismemberment – look more impactful (like time slowing down and the camera zooming in communicating that something important has just happened).

Visual feedback is also important to the feel of a game, as every action the player takes should have a noticeable effect on the game world and its state. Players dislike when their actions are not being acknowledged by the game [4].

3.3 Enemy AI Based on Body Parts

The behavior system needs to provide a way to define different behaviors depending on the current game state while being able to efficiently and cleanly change from one behavior to another, such as "Fly Up", "Descend", "Rake with Claws", etc. The finite state machine can become difficult to manage when facilitating more complex *adaptive* behavior like an enemy adapting to its injuries, having to define the various state transitions and their conditions. The behavior tree (BT) is a cleaner solution for the decision-making and adaptability requirements of this use case.

3.3.1 Behavior Tree

The behavior tree has a GUI (graphical user interface) to help streamline not only the creation of BTs, but also their fast editing and iteration. The node-based architecture of Godot is well-suited for the node-based structure of BTs, and thus the BT can live inside the enemy scene as a Root node and its collection of BT child nodes. It is constructed when the enemy node is loaded into the scene via the `_ready()` function that runs when a node and its children have entered the scene tree.

The implementation can use Godot's signals for event-based traversal and execution logic. Instead of the branch constantly calling a `tick()` function, the parents can just connect to the children's `finished` signal and react when it is emitted. The signalization is done alongside a *return status* passed with the signal emission.

The behavior tree has a unified interface so that each node knows how to interact with all the other nodes. Aside from normal nodes that can have any number of children, decorators provide specific behavior attached to a single node (as per the *decorator* pattern). The behavior tree also needs all the basic nodes for traversal – root, sequence, selector, and the condition decorator. The action node is another essential node, serving as the leaf node where the actual behavior that a player sees is executed.

A blackboard is essential as a shared data storage that all of the nodes have access to and can communicate through.

When considering the more specialized nodes, the goal of adaptability needs to be taken into account, and with that – nodes that can interrupt the current behavior being executed. A priority-based reselector is useful for interrupting the running branch when one with a higher priority becomes available. Behavior can also be interrupted when a body part is destroyed, as described in Section 3.3.2.

Nodes that implement an element of randomness can break up the predictability of an enemy's behavior and engage the player more. The simplest of these are the ones that choose which child to execute completely at random and ones that decide randomly, but through weighted probability. An advanced weighted randomization is the utility node, which calculates the probability weights of its children based on the variables of the current game state, and then decides randomly based on those weights.

3.3.2 Limiting Behavior

The behavior tree takes into account the state of the enemy's body parts and reacts to it appropriately. This feature is implemented by the two body part filter decorator nodes. One checks once and executes its child or fails, and the other keeps checking continuously, interrupting the current behavior if a required body part is destroyed. This effectively culls the branches of the behavior tree that can no longer be performed due to missing body parts, limiting the enemy's options.

3.4 Knockback

When the player hits the enemy with an attack, the enemy reacts with a jerk to provide the player with visual feedback through the enemy's reactivity. A knockback can be implemented by adjusting the bones of the enemy skeleton rig. The system needs to be able to detect which body part was hit, which is possible by separating the hitboxes of each body part, and then it needs to gradually offset the bone position of that body part and return it back to normal.

3.5 Mesh Deformation

Another way of adding reactivity to some of the more powerful player attacks is to add mesh deformation to the attack's hit location on the enemy body. Such deformation is achieved via a vertex shader that displaces vertices around the hit location, which is passed as a parameter to the shader. The deformation strength and radius are adjustable for each projectile. A magnitude mask on the mesh (encoding how much or how little a vertex should be moved when deforming) prevents extreme or awkward deformation in areas sensitive to vertex displacement.

3.6 Enemy Design

A manticore was chosen as the enemy for the demo because it offers multiple possibilities for body-part-based abilities (biting, claws, legs, wings, or a stinger). These body parts can be combined as requisites for different movement and attack behaviors, allowing the player to disable them when they destroy the body parts. They are split into separate meshes so that their visuals can be controlled individually. Each behavior has an animation, including hindered movement animations where the manticore is missing body parts.

3.7 Player Controller

Depending on the implementation of the player controller, there are even more elements of player choice and rules complexity. Such cases could include the player maneuvering around the virtual space to avoid or close in on an enemy, or any of the player's special abilities and attacks with various cooldowns and effects. However, the player controller is a complex issue of its own. It is not covered by this thesis, but the player's experience is a crucial aspect of games and should be accounted for when discussing game design.

This demo's player controller implements basic movement and camera controls and three types of attacks – slashing (arc), piercing (point), and bludgeoning (larger shape), each with their own models, hit detection, visual effects and decals.

Chapter 4

Implementation and Demonstration

4.1 Modeling

Due to the unique requirements of the enemy model for this game demo, a custom model was designed and created. A simple arena-like desert basin surrounded by mountains was also modeled. Both models were made in Blender.

4.1.1 Enemy Model

The enemy model contains separate body part meshes, joined together as a cohesive whole. Each mesh has its own surface material as well, so that it can be controlled directly and in isolation from the other parts in the game engine.

First, a detailed model of a manticore was sculpted as a whole. Due to the high vertex count left after the sculpting workflow, the model was retopologized (simplified into fewer polygons) while maintaining the proper shape and details, but being more performance-efficient. The model was then UV-unwrapped (the vertices mapped onto a 2D image) and textured. Once the base model was finished, the skeleton rig was made using the Rigify Blender addon. Rigify's quadruped template with wing bones taken from the bird template was used. The rig was edited to fit the manticore, and some bones were added (such as the tail bones). Blender generated automatic weight paints (which determine how much a bone deforms the mesh), although they had to be manually adjusted in some areas.

Finally, the model was separated into the different body parts that would be targetable by the player – the head and torso, the legs, the wings, and a stinger tail. The vertices were ripped at the seams and the left over holes filled with new geometry to seal them.

Animations were made in Blender using the skeleton rig's inverse kinematics to position the bones and deform the mesh and then capturing these positions in the keyframes of the animation. These animations were catered to the different usage of body parts for the various types of movement and attacks that the manticore enemy possesses. Movement animations include the idle animation, walking, running, running with a limp, crawling, walking on hind legs, leaping, flying, and diving. Attack animations include biting, raking with claws (left, right, or both), and a stinger tail attack.

The whole model then had its vertices painted, with the red channel signifying the deformation magnitude (see Section 4.3). Controlling the deformation magnitude in sensitive areas can be crucial to preventing the revealing of body part seams or extreme deformations



Figure 4.1: The manticore model with detached body parts, rendered with vertex colors.

where they are not desired. Notice the black vertex colors where the body parts separate in Figure 4.1.

4.1.2 Environment

A sandy desert basin arena was modeled to test the enemy behavior in. To avoid the visible repetition of the seamless textures applied to the terrain, two texture shaders were developed that reduce these pattern artifacts. For the sand shader (displayed in Algorithm 4.1), the same texture was mixed at a different scale and rotation, the results of which can be seen in Figure 4.2. The rotation calculation uses the standard 2D rotation matrix.

```
in: float scale_1, float scale_2, float angle_offset

void fragment() {
    vec4 albedo_1 = texture(texture_albedo, UV * scale_1);
    float angle = radians(angle_offset);
    vec2 centered_uv = UV - 0.5;
    vec2 rotated_uv = vec2(
        centered_uv.x * cos(angle) - centered_uv.y * sin(angle),
        centered_uv.x * sin(angle) + centered_uv.y * cos(angle)
    );
    vec4 albedo_2 = texture(texture_albedo, rotated_uv * scale_2);
    ALBEDO = mix(albedo_1, albedo_2, 0.5).rgb;
}
```

Algorithm 4.1: Sand shader – repeating the same texture with a different scale and rotation.

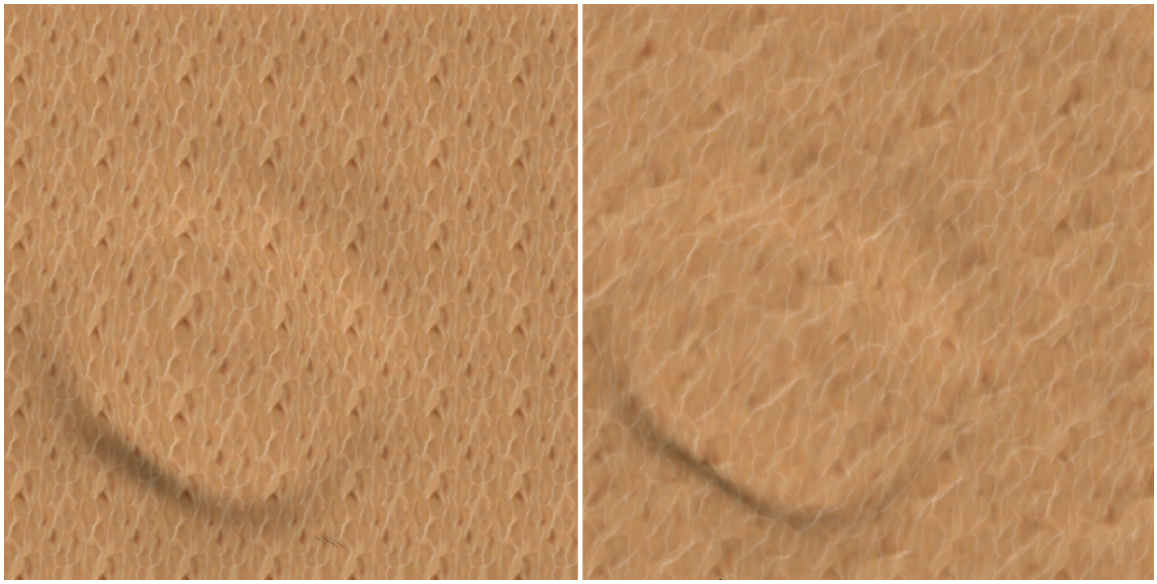


Figure 4.2: The sand environment – simple texture repetition (left) vs. shader (right).

A slight vertex displacement based on the height map texture of the sand was also added to the shader to roughen the unnaturally smooth ground mesh. The shader code in Algorithm 4.2 was shortened, including omitting the already stated parameters. The `rotated_uv` calculation is equal to the one in Algorithm 4.1, and the same scale and rotation is applied so that the height texture matches the mixed albedo texture.

```
in: sampler2D height_map, float max_height_offset

void vertex() {
    float height_1 = texture(height_map, UV * scale_1).r;
    height_1 = height_1 * 2.0 - 1.0; // normalize to -1 to 1
    float height_2 = texture(height_map, rotated_uv * scale_2).r;
    height_2 = height_2 * 2.0 - 1.0;
    float height = mix(height_1, height_2, 0.5);
    // move vertices up or down on the y axis
    VERTEX += vec3(0.0, 1.0, 0.0) * height * max_height_offset;
}
```

Algorithm 4.2: Sand shader – vertex displacement based on the height texture.

For the mountains, two different textures were mixed based on noise and slight color darkening, strongest at the transitions, was applied to further break up the monotony.

The shader mixes the two albedo textures based on the seamless noise texture. For the transition regions, we can figure out the extremes (values 0.0 being fully texture 1, or 1.0 being fully texture 2) by subtracting 0.5 from the noise texture, the values of which span 0 to 1. That maps the absolute values to 0.0 (the transition point) to 0.5 (purely textured by one of the textures). Then we map it back to 0 to 1 by multiplying it by 2 and invert it to get the transition regions (with 0.0 being no transition), as seen in Algorithm 4.3.

```
in: float scale_1, float scale_2, sampler2D texture_albedo_2, sampler2D
    texture_normal_2, sampler2D noise, float noise_scale

void fragment() {
    float noise_value = texture(noise, base_uv * noise_scale).r;
    vec4 albedo_tex_1 = texture(texture_albedo, base_uv * scale_1);
    vec4 albedo_tex_2 = texture(texture_albedo_2, base_uv * scale_2);
    // mix the 2 albedo textures based on noise value
    vec3 mixed_albedo = mix(albedo_tex_1, albedo_tex_2, noise_value).rgb;
    // calculate transition regions
    transition = 1.0 - abs(noise_value - 0.5) * 2.0;
    // add some darkening along the transitions for more variation
    ALBEDO = mix(mixed_albedo, mixed_albedo * vec3(0.5), transition);
}
```

Algorithm 4.3: Rocks shader.

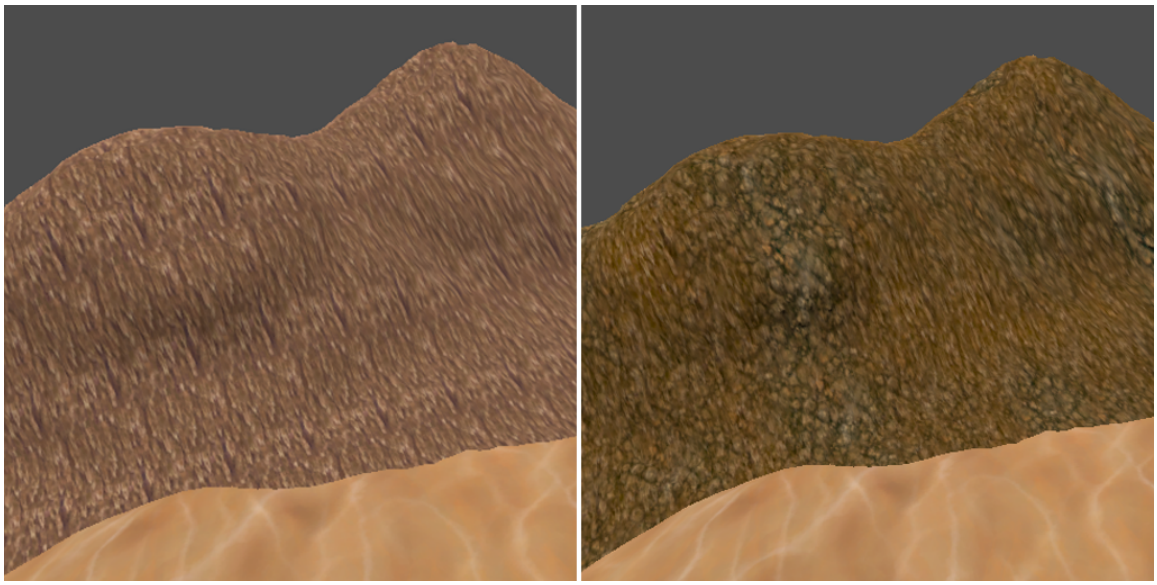


Figure 4.3: The rocks – simple texture repetition (left) vs. shader (right).

All environment textures were taken from StylizedTextures[1]. The skybox was taken from FreeStylized[2]. Both websites offer their works under the CC0 license as of the writing of this thesis.

4.2 Behavior Tree

The behavior tree (BT) is the central control unit of the enemy AI. It decides the flow of the decision-making and behavior execution via its node system. Each BT node inherits from

[1]Stylized Sand 002, Stylized Rocks 001, Stylized Cliff Rock 006, available at: https://stylizedtextures.com

[2]Skybox 24, available at: https://freestylized.com

```
@abstract class_name BTNode

signal finished(status: ReturnStatus)

enum ReturnStatus { RUNNING, SUCCESS, FAILURE }

var blackboard: Blackboard
var child_nodes: Array[BTNode] = []

func construct_tree(bb: Blackboard) -> void:
	blackboard = bb
	var scene_child_nodes = get_children()
	add_children_and_call_their_construct()
	for child in child_nodes:
		child.finished.connect(_on_child_finished)

# to be overridden by subclasses
func _on_child_finished(_return_status: ReturnStatus) -> void:
	pass

# cleanup func for interruptable nodes; to be overridden by subclasses
func abort() -> void:
	pass

@abstract func execute() -> void
```

Algorithm 4.4: The `BTNode` class which all behavior tree nodes inherit from.

the abstract class `BTNode`, the implementation of which can be seen in the GDScript pseudocode Algorithm 4.4. It is also where the return status enumeration is defined. Currently, due to the event-based architecture, only `SUCCESS` and `FAILURE` are used, but `RUNNING` is also included should it ever be useful.

An event-driven behavior tree was implemented over a tick-based one (see Chapter 2). It allows the branches to manage their own uptime and just emit the `finished` signal when they are done, letting the parent nodes that are subscribed to that signal to handle the exit. Signals in Godot are equivalent to `events` in languages like C#.

To share information among the various branches, the BT uses a `Blackboard`, which is a dictionary that every BTNode has access to, initiated in the root and propagated to every node.

Each BTNode can also implement the `abort` function, which is used to interrupt currently running behavior. It is the responsibility of each node to correctly abort its currently running children. This abort logic has to propagate from the node that caused it all the way through to the currently executed node farthest down the behavior tree, so any nodes that control the flow of the tree, as well as the decorators, must implement it.

The `BTDecorator` class that all decorator nodes inherit from is equal in functionality to `BTNode`, but allows only a single child node.

Each behavior tree requires the `BTRoot` as the root node. All the other nodes must be added as its descendants.

4.2.1 Node Types

This section describes the implementation of all the different behavior tree nodes.

Root

The root node is the first node of any behavior tree. It is responsible for the construction of the whole tree and the initialization of the blackboard. It executes its children sequentially, no matter the return state. The execution loops back to the first child node when the last child finishes.

Sequence

The sequence node executes its children in order. If any of them finish with `FAILURE`, the whole sequence node finishes with `FAILURE`.

Selector

The selector node executes its children in order. If a child node finishes with `SUCCESS`, the whole selector node finishes with `SUCCESS`. Otherwise, it continues to the next child node. It finishes with `FAILURE` if none of the children succeed.

PriorityReselector

The priority reselector node acts like a selector node, but continuously (in the defined time interval) checks the availability of child nodes that are in the node tree before the one that is currently being executed. This rechecking only happens for child nodes that are `Condition` decorators (their condition is the one `PriorityReselector` checks against). In case a previous node becomes available (the condition result returns as true), the priority reselector aborts the current child and starts executing the newly available child.

For example, an `AttackOrIdleReselector` has a `Condition` child that checks if the player is seen by the enemy and an `Action` node which performs the Idle behavior. Normally, the enemy executes the Idle action indefinitely, but when the player comes into the enemy's detection range, the priority reselector now evaluates the first `BTCondition` child as true, aborts Idle, and starts executing the attack branch.

Condition Decorator

The condition decorator executes its child node only if its condition evaluates as true. It contains a condition function, which is called whenever this node is executed. If it returns true, it executes its child and is ready to handle its return status as normal. Otherwise, the condition decorator finishes with `FAILURE`.

This can be used to filter out certain branches based on the given conditions and is generally useful in controlling the decisional flow of the tree traversal.

Randomizer, WeightedRandomizer, Weight Decorator

These nodes serve to randomize the flow of the behavior tree.

`Randomizer` is a pure randomization node. It chooses which of its child nodes to execute completely at random.

`WeightedRandomizer` chooses a random child to execute based on the weights of the child. It expects the `Weight` decorators, which have a settable float `weight` property, as direct children; otherwise, the default weight is 1.0.

Both randomizer nodes account for body part filters, and do not consider branches that are filtered out due to missing body parts in the randomized child selection.

UtilityRandomizer, Utility

Utility randomizers work much like weighted randomizers, except the weights are dynamically calculated via utility functions that often use the variables of the current game state to evaluate which branch would be the most valuable to execute.

The randomization also heavily relies on the type of random number generator used. A generator with a continuous uniform distribution will maintain pretty even randomization. On the contrary, if a normal distribution were to be used, the randomization would be heavily skewed towards some of the choices. This might be desirable if the utility weights themselves are not sufficient in altering the probability towards choices with greater weights and the developer wishes to further alter the probability scale. However, this project uses a continuous uniform distribution generator to preserve an even level of randomness of the chosen branches.

Utility calculations can bring out the most dynamic behavior in enemy AI, as they directly work with the state of the game to calculate the usefulness of a branch. It mainly depends on the complexity of the calculations and the game state variables taken into account.

Cooldown Decorator

The cooldown decorator only executes its child when the cooldown is available. Once the child node is executed, a cooldown timer is started, for the duration of which the decorator finishes with `FAILURE` if it is executed again. Once the timer runs out, the child returns to its default usable state.

BodyPartFilter, BodyPartFilterRecheck

The body part filters are decorators that filter out branches where body parts are unavailable due to having been destroyed. They have an array of all required body parts and an array of body parts where at least one is needed to not be filtered. When executed, the decorator finishes with `FAILURE` if any body part is unavailable; otherwise it normally executes the child node. The `BodyPart` class is described in Section 4.4.

`BodyPartFilter` is appropriate for one-and-done checks, while `BodyPartFilterRecheck` is better for branches where a behavior is being executed over a longer period of time. Since it continuously checks the available limbs and aborts and finishes with a `FAILURE` when a body part becomes unavailable, it is useful for detecting injuries in the middle of action execution.

Action

Action nodes are leaf nodes for the standard behavior execution. This is where the code for the behavior that affects the game world is implemented, such as the pursuit of the player or the bite attack.

Blackboard

The blackboard is a shared data storage that every behavior tree node has access to. It is implemented as a dictionary with the `store`, `read`, and `remove` functions that work with keys and values of the `Variant` type (meaning any type can be used).

4.2.2 Creating Behavior Trees inside the Scene Tree

Developing a behavior tree only via code can be confusing and unwieldy. Having a graphic representation in which they can be edited can bring clarity to their structure (since the parent-child hierarchy is clearly visible) and make their creation and iteration faster.

Godot's node architecture is well-suited for this purpose. Adding behavior tree nodes to the scene tree can be an intuitive way of construction. Behavior tree nodes can be created by defining their scripts as classes with `class_name` at the top – they automatically become nodes that can be added to the scene tree. Each node must then have an implementation of the `construct_tree` function (obtaining the children from the scene tree and calling the construct function for each of its children) and a unified interface that can be relied on when driving the flow of the behavior tree (such as the `execute` and `abort` functions). When the enemy is loaded into the scene, the `Root` node gets all of its children from the scene tree and starts the behavior tree construction. Once that is finished, it starts executing the behavior tree, so the enemy AI starts immediately. The root script is where any starting variables should be added to the blackboard.

This way, the behavior tree structure can be constructed inside the editor. Each node can then be extended with a script that defines the necessary behavior (such as adding a condition function to the `Condition` decorator or attack logic to an `Action` node).

For visual clarity, icons were created to differentiate the node types at first glance, as can be seen in Figure 4.4.

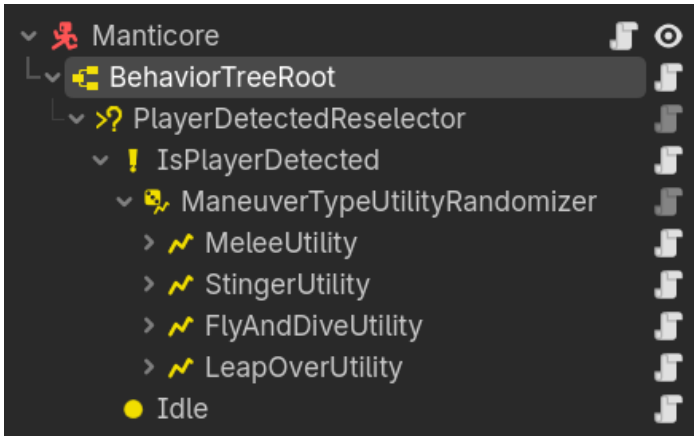


Figure 4.4: The behavior tree nodes inside the enemy scene tree.

4.3 Deformation Shader

When a projectile with deformation parameters hits the enemy, the body part handles it and sends the necessary data to its shader. Each body part shader uses a `StandardMaterial` converted to a `ShaderMaterial` – a `spatial` shader which handles everything the material did, like the albedo textures, normal textures, etc. Deformation code was added manually and is delimited by comments, and it can be seen in Algorithm 4.5.

The shader's uniform variables are an integer `hit_count` and arrays containing data for each hit – `hit_positions` and `hit_normals` (both in the mesh's local space), and the `hit_radius` and `hit_strength` arrays (containing the projectile's deformation parameters). The total capacity is 16 hits; when overflowed, the oldest hit is removed.

In the `vertex()` function, which is able to control the mesh's vertices, the shader loops over the hits. For each hit up to `hit_count`, it measures the distance of the hit position to every vertex (denoted by the `VERTEX` variable). If that distance is lower than the hit's `hit_radius`, it calculates the normalized distance from the hit position to the vertex, called `falloff`, and smooths it by squaring it, so that the farther the vertex is from the hit, the less it is displaced.

The vertex is then offset away from the hit (along the inverted normalized `hit_normal`), scaled by the falloff, the projectile hit's `hit_strength`, and the red channel of the vertex color (`COLOR.r`).

The red channel of the mesh's vertex colors acts as a magnitude mask, and it affects how much a given vertex should be displaced. This concept is similar to the weight paints of a skeleton rig, which is a sort of "heat map" that modifies how much a given bone's transformations deform the mesh in each point of the mesh.

This vertex displacement modifier mask was necessary due to the need to author the deformation on different parts of the mesh – an equal magnitude of mesh deformation is not desired in all parts of the mesh. A granular control via vertex colors offers a more authored enemy design, and prevents awkward deformations in sensitive areas, an example of which can be seen in Figure 4.5.



Figure 4.5: The same magnitude of deformation for every vertex of the mesh can produce undesirable results.

```
in: int hit_count = 0, vec3 hit_positions[16], vec3 hit_normals[16], float
    hit_radius[16], float hit_strength[16]

void vertex() {
    float deformation_magnitude = COLOR.r; // vertex color
    for (int i = 0; i < hit_count; i++) {
        float radius = hit_radius[i];
        float strength = hit_strength[i];
        float dist = distance(VERTEX, hit_positions[i]);
        if (dist < radius) {
            float falloff = 1.0 - (dist / radius);
            falloff = falloff * falloff; // smooth
            VERTEX -= normalize(hit_normals[i]) * falloff * strength *
                deformation_magnitude;
        }
    }
}
```

Algorithm 4.5: Deformation shader.

`BodyPart` duplicates the mesh's active material (to make it unique from other enemies of the same type using the same material) and initializes the shader parameters. Whenever a body part receives a hit, it is appended to the `hit_locations` array. When more than 16 hits are stored, the oldest is removed. Hits that are too close to each other are discarded to prevent awkward and extreme deformation. Each hit has a bone attachment that it is attached to (taken from the `BoneAttachment3D` node that the hitbox is attached to), the local space of which is used for the hit position and normal vectors. The attachment, position, normal, radius, and strength are then added to the `hit_locations` array.

Due to bone movements, the positions of the hits need to be recalculated every frame. The `update_deformation_shader()` function iterates through the `hit_locations` array and first composes an attachment-to-mesh transformation matrix as the mesh's inverse global transform multiplied by the current bone attachment global transform. It then converts the hit position with this matrix and converts the hit normal with the inverse transpose of the matrix's basis (using $n' = (M^{-1})^T \cdot n$). The converted position and normal, as well as the hit's radius and strength, are then sent to the shader uniforms.

4.4 Body Part Nodes

The logic for the separated body parts is contained within the `Body`, `BodyPart`, and `Hitbox` classes.

Like the behavior tree nodes, these classes are also meant to be added to the scene tree (see Section 4.2.2), with `Body` being the root node and `BodyParts` being their direct children. `BodyPart` nodes should then contain `BoneAttachment3D` nodes (that copy the transforms of the specified bone onto their children, making them move with the bone) with `Hitboxes` that contain the desired collision shapes. An example chain of nodes would be: `Body` → `LegFL (BodyPart)` → `ThighFLBone (BoneAttachment3D)` →
→ `ThighFLHitbox (Hitbox)` → `CollisionShape3D`.

There can be several `BoneAttachment3D` nodes, each with its own `Hitbox`, to detect hits more accurately (since collision shapes are usually simple shapes like capsules).

4.4.1 Health Management

The `Body` class tracks the central amount of health the enemy has, which determines whether it is alive or dead. `BodyParts` have their separate health pools, which determine whether that body part is destroyed or not, but they also have an option to make damage bypass their own health and only damage the central health pool. That option is useful for body parts that themselves cannot be destroyed, but contribute to the overall defeat of the enemy, such as the torso and head.

When a body part reaches 0 health, a notifying signal is emitted. A special camera effect plays and time slows down (described in Section 4.6), the separated mesh turns red for a moment and then the mesh is hidden, the bones and hitboxes freed, and a severed body part scene is spawned in the destroyed part's location with physics independent of the enemy.

When the body reaches 0 health, a notifying signal is emitted, the behavior tree is stopped, and the ragdoll is activated (further described in Section 4.7.4). After a moment, the manticore's body fades into full transparency and is freed.

4.4.2 Hitboxes and Hit Management

`Hitbox` nodes inherit from `Area3D` nodes, which are able to detect when another area or body (like a projectile) collides with it. The hitboxes receive the hits with parameters wrapped in a `ProjectileHitData` resource. It contains `damage_amount`, `world_position`, `world_normal`, `projectile_direction`, `decal_params`, and `deformation_params`. The world position and normal of the detected hit are then converted to the local space of the bone that the hitbox is attached to. Then the hitbox emits a signal with the original parameters, except with the world-space hit position and normal replaced for the converted ones.

`BodyPart` nodes are connected to this signal and whenever it is emitted, they start processing the hit. The global `enemy_hit` signal is emitted. Damage is applied to the body part (based on the hit's `damage_amount` parameter), the body part's flash effect is played, and the hit decal is added to the body part (both described in Section 4.4.3). If the body part has a knockback node attached, it applies the knockback using the `projectile_direction` vector (see Section 4.5). Finally, the hits are registered into the deformation arrays that are supplied to the deformation shader (see Section 4.3).

4.4.3 Hit Effects

When the body part is hit, the flash effect is achieved via simple fragment shader code that controls the material's `emission`, interpolating between 0 and the flash strength, and back to 0.

Hit decals are spawned, attaching a `Decal` node oriented by the hit's normal to the `BoneAttachment3D` parent of the hitbox that was hit. These nodes project textures onto surfaces that they intersect with, hence why it is important to size and orient the node correctly.

When a body part is destroyed, it starts a continuous spewing of blood particles. The particles themselves, implemented via the `GPUParticles3D` node and a custom blood tex-

ture, are spawned at a predetermined position and rotation on the enemy's body, activated by a `BodyPart`'s `body_part_destroyed` signal. The blood material's properties are set to `transparency: alpha scissor`, which, in combination with alpha curves that control the transparency of the material over time, makes it look like the blood congeales into separate strands as the alpha value decreases.

4.5 Knockback `SkeletonModifier3D`

The skeleton has a custom `BodyPartKnockback3D` node attached to it. The class inherits from `SkeletonModifier3D` which allows developers to modify skeleton deformations via code. It applies bone modifications after animations (if an `AnimationMixer` is present), and they can even be stacked to modify the skeleton one after another as necessary.

Each `BodyPart` can have a custom `BoneKnockback3D` node attached to it. The node's export variables are the bone to apply knockback to and the strength of the knockback. The `apply_knockback` function is called from `BodyPart` whenever a hit is received. `BoneKnockback3D` then converts the world-space projectile direction into the skeleton's local space, as that is the coordinate system that `SkeletonModifier3D` works with. The bone to apply knockback to, the direction, and the strength are then sent to the `BodyPartKnockback3D` node to be added to the ongoing knockback effects.

In `BodyPartKnockback3D`, the new knockback is added to the `active_knockbacks` dictionary where each entry is tied to a bone index key and contains the target offset, the current progress, and a bool flag signifying whether it is getting to the target offset or returning back to a zero offset. It also adds a knockback to the base bone of the whole body (usually the base of the spine chain), which is defined as an export variable in the class.

The `_process_modification_with_delta` function from `SkeletonModifier3D` is used to process the knockbacks. Inside it, the active knockbacks are iterated through and their progress towards their target offsets is advanced by the defined speed. The current offset is applied on top of the skeleton's animations as an interpolation between a zero vector and the target offset based on the current progress (0.0 to 1.0). Once a target offset is reached, the `is_returning` flag is switched on and the progress is interpolated back towards a zero vector. After it reaches progress 0.0, it is removed from `active_knockbacks` entirely.

4.6 Player Controller

The player controller implements a simple movement script from Godot's `CharacterBody3D` template. The player can choose between 3 types of attacks by pressing `1`, `2`, or `3`.

Each projectile has several export variables that can be set. The projectile's type can be slashing (a wide arc), piercing (a point), and bludgeoning (a sphere). This affects the way it detects hits. Slashing projectiles project a number of raycasts along its width, and the middle raycast's hit is preferred. Piercing projectiles have a single raycast from their center. Bludgeoning projectiles use a shapecast (an analogue to a raycast, but uses a shape instead of a line). Each hit's position and normal is recorded and handed off to the `Hitbox` that was hit. Next, the projectile's damage amount and travel speed can be set, as well as whether it should be affected by gravity. Each also contains decal parameters that are to be spawned on impact and attached to the enemy (see Section 4.4.3), impact visual effects (the projectile's break effect and blood particles that are spawned on a hit), and

deformation parameters (see Section 4.3). For example, the boulder attack is affected by gravity, deforms the enemy's mesh, leaves a bruise decal, and spawns rock fragments and dust particles on impact.

When a body part is destroyed, the player is notified via the `body_part_destroyed` global signal. The time scale of the game is momentarily lowered so that it slows down, and the player camera zooms in.

When the player hits an enemy, an X-shaped crosshair appears to communicate the hit to the player. When the player is hit by an enemy, a blood texture flashes on the edges of the screen. Numerical damage to the player and the player defeat state was not implemented, as it is not the focus of this thesis.

4.7 The Manticore Enemy

The manticore enemy is a `CharacterBody3D` node that implements the described systems as child nodes. It has the skeleton rig which contains all of the body part meshes and the knockback skeleton modifier. The `AnimationTree` allows control over playing animations. Two movement collision shapes for the front and back determine its collisions with the environment. An `Area3D` node notifies if a player has entered or exited its detection area. `MovementParticles` and `SeveringBloodParticles` are parents to nodes in the 3D space that mark where these particles should spawn. The behavior tree drives the enemy decision-making and actions. And finally, the `Body` and its `BodyPart` children that keep track of the body parts' state, and their `BoneKnockback3D` children that contain the knockback information passed to the knockback skeleton modifier.

4.7.1 Body Parts

The manticore's body contains the legs, the wings, the tail, and a head-and-body part. Each body part usually has many `hitboxes` to accurately detect collisions and a single `BoneKnockback3D` node to add reactivity to hits. Each also has the `RigidBody3D` scene of the severed version of itself which then spawns in its location and applies physics independent of the enemy body. All of the body parts except for the head-and-body one, which is the only node that subtracts from the central health pool, can be destroyed. These body parts are used within filters inside the behavior tree.

4.7.2 Behavior Tree

The manticore is in the `Idle` state until the `IsPlayerDetected` condition becomes true, which the Reselector notices during its rechecking and switches to the higher-priority branch. Then the enemy chooses from its different maneuvers based on distance-based utility calculations. When farther from the player, the utility of a ranged stinger attack or the fly and dive maneuver is higher than that of a melee attack, and vice versa. Maneuvers are filtered out from the randomized selections if they are unavailable due to missing body parts via the body part filters. See Figure 4.6 for the behavior tree hierarchy.

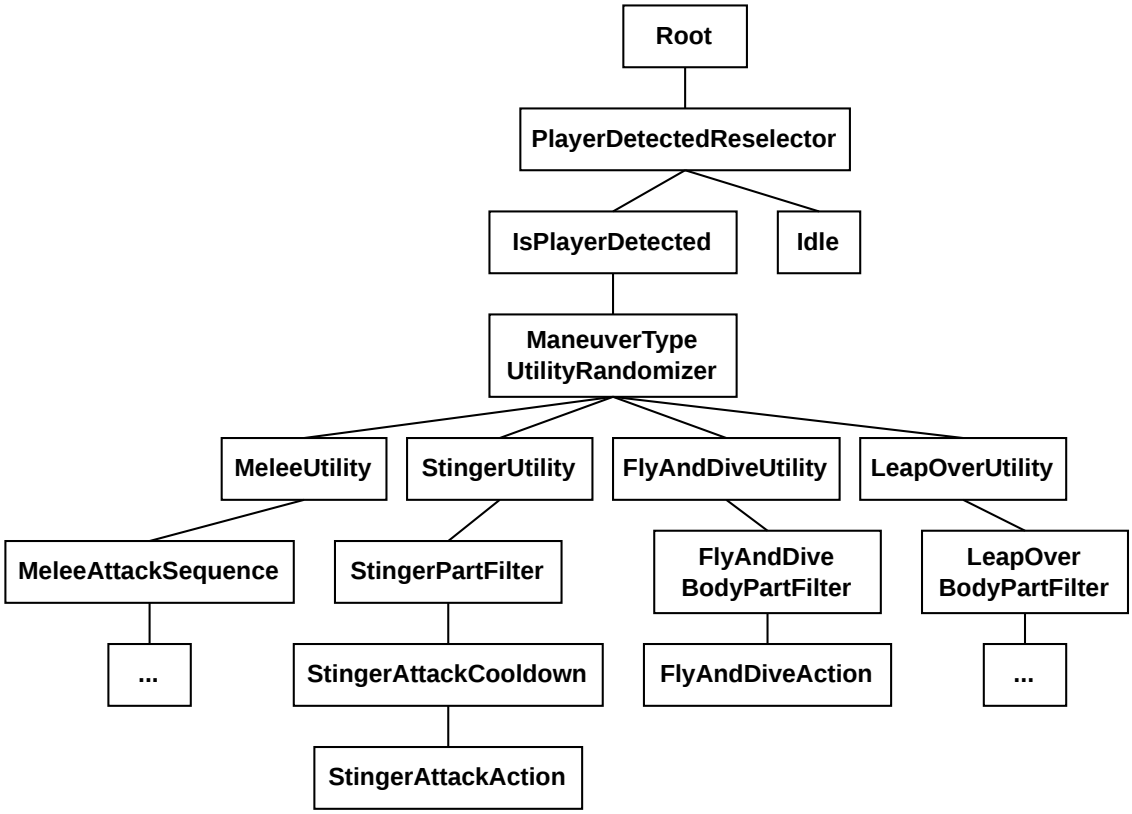


Figure 4.6: The base of the manticore's behavior tree.

In the melee attack sequence, the enemy first pursues the player, as can be seen in Figure 4.7. A selector tries to execute the types of movement in order (each fallback with more debilitated body parts than the last). It succeeds if the enemy comes within melee range of the player, and the parent sequence continues. While pursuing the player, the body part filters continuously check whether the necessary body parts for that type of movement are available. If any are destroyed while in pursuit, the filters detect it, abort the movement, and emit a failure. The parent selector then continues to the next fallback. If no movement options are available, the enemy activates its ragdoll mode (described in Section 4.7.4). This fallback behavior further facilitates the manticore's dynamic adaptability to its injuries.

4.7.3 Animation Tree

The `AnimationTree` node contains all of the manticore's imported animations, as well as controls to switch which animation is playing. Most of the animations use `BlendTree` nodes that create a single chain from the first input all the way to the animation output. Blend trees are able to gradually blend between animations thanks to their blend amount value. In code, this value is interpolated between 0.0 and 1.0 over time, creating a smooth transition between the animations. Attack animations are done via the `OneShot` node and an `AttackStateMachine` that chooses between the different one-shot attack animations based on transition variables set from code.

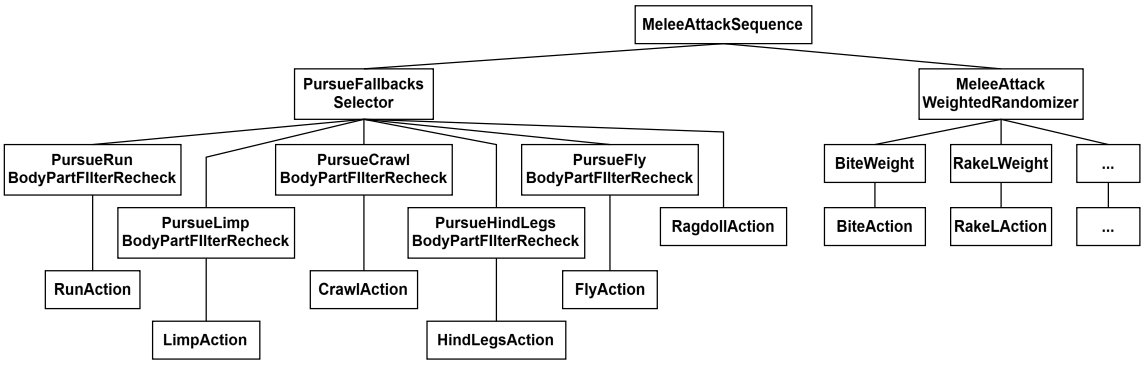


Figure 4.7: The manticore's melee attack sequence nodes with different movement fallbacks based on missing body parts.

4.7.4 Visuals, Adjustments, and Effects

There are many small visual adjustments and effects that contribute to the natural feel of the enemy.

The manticore's model is rotated towards the player. To avoid the rotation being constant and instantaneous, the rotation always has a speed with which it interpolates towards the player. The rotation is also periodically locked for a random time window to allow the player to better maneuver around the enemy. The target rotation around the y axis is calculated as the arctangent of the x (the side axis) and z (the forward direction) components of the vector pointing from the manticore to the player (which yields the angle of that vector in the xz plane). That rotation is then used in `rotate_toward()` to interpolate the current rotation towards the target rotation using the turn speed variable.

The alignment to the ground is done via two raycasts from the manticore's ground colliders. The normals of the hits are averaged and used to calculate the rotation of the manticore's model. The rotation on the z axis is dampened to give the manticore a more natural upright tilt.

When the enemy moves on the ground, sand particles are spawned at the feet with a slight random displacement. The particles are only spawned if the enemy reaches a speed threshold. The particle node also keeps track of whether the enemy is in flight and spawns impact particles when it lands.

The visual effects when the enemy is hit are described in Section 4.4.3.

The ragdoll is activated when the enemy is killed or when the behavior tree activates it when no movement options are available. The ragdoll uses `PhysicalBoneSimulator3D`, generated from the skeleton rig, to simulate the physics of the enemy's body parts. The joints' type is set to `6DOF` which gives the ragdoll a more natural feel.

In various classes, the export variables are shown or hidden based on other variables (usually based on boolean flags or chosen enum options). For example, in the `BodyPart` class, the export variables are shown or hidden based on the `remove_part_if_destroyed` variable, as shown in Figure 4.8. When using a skeleton, the bone options are supplied after the skeleton is loaded as a list of bone names. When the type of projectile is chosen, the relevant export variables are shown and hidden accordingly.

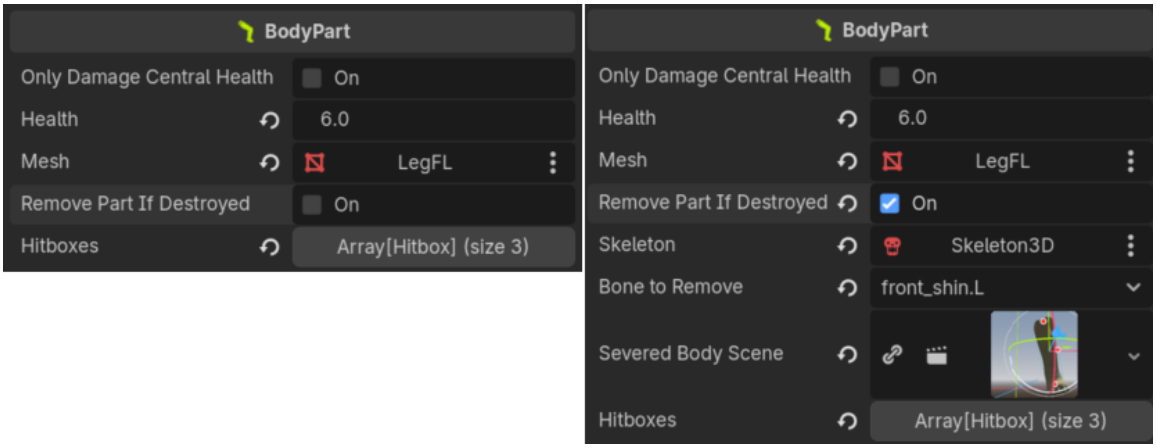


Figure 4.8: The export variables are being shown or hidden in the Godot editor depending on the value of `remove_part_if_destroyed`.

4.8 Demonstration in Various Test Cases/Situations

TODO

Chapter 5

Performance Testing

Godot profiler

Shader Precompilation - lag spikes whenever a new effect is on screen, like a new projectile or the manticore body transparency triggered - loading screen while all of the effects are played in the background to „precompile“ the shaders

Chapter 6

Documentation, Publication, Conclusion

Bibliography

[1] Adams, E. and Dormans, J. *Game Mechanics: Advanced Game Design*. New Riders, 2012. Voices That Matter Series. ISBN 9780321820273.

[2] Aristidou, A.; Lasenby, J.; Chrysanthou, Y. and Shamir, A. Inverse kinematics techniques in computer graphics: A survey. *Comput. Graph. Forum* online. Wiley, September 2018, vol. 37, no. 6, p. 35–58. ISSN 0167-7055. Available at: https://doi.org/10.1111/cgf.13310. [cit. 2026-01-08].

[3] Graham, D. A Reusable, Light-Weight Finite-State Machine. In: Rabin, S., ed. *Game AI Pro 3* online. 1st ed. CRC Press, June 2017. ISBN 9781498742580. Available at: https://www.gameaipro.com/GameAIPro3/GameAIPro3_Chapter12_A_Reusable_Light-Weight_Finite-State_Machine.pdf.

[4] Orselli, B. Gabe Newell: realism isn't fun, I play games to have fun. *Niche Gamer* online. 20. November 2023. Available at: https://nichegamer.com/gabe-newell-realism-isnt-fun-i-play-games-to-have-fun/. [cit. 2026-01-20].

[5] Rabin, S. *Game AI Pro: Collected Wisdom of Game AI Professionals* online. 1st ed. CRC Press, 2013. ISBN 9781466565975. Available at: https://www.gameaipro.com.

[6] Schell, J. *The Art of Game Design: A Book of Lenses*. CRC Press, Taylor & Francis Group, 2019. Computer graphics / Game design. ISBN 9781138632097.